
MindHarbor

Plateforme de soutien en santé mentale

Examen de mi-session

Hackathon 88 heures

Début	Jeudi 13 août 2026 à 8 h
Fin et remise	Dimanche 16 août 2026 à 23 h 59
Durée	88 heures (4 jours)
Équipes	2 à 6 personnes
Pondération	100 points (+ 10 points bonus)
Remise	Fichier REMISE.md déposé sur Teams

Technologies imposées

TypeScript · Node.js · Express · React · Axios · Prisma · Neon PostgreSQL · JWT

Table des matières

1 Informations générales	3
1.1 Cadre de l'évaluation	3
1.2 Objectifs pédagogiques	3
1.3 Pile technologique imposée	4
1.4 Architecture attendue du dépôt	4
2 Le projet MindHarbor	5
2.1 Contexte	5
2.2 Acteurs du système	6
2.3 Fonctionnalités du noyau obligatoire	6
2.3.1 A. Journal de bien-être	6
2.3.2 B. Analyse et tendances	7
2.3.3 C. Ressources et exercices	7
2.3.4 D. Groupes de soutien	7
2.3.5 E. Messagerie privée et confidentialité des échanges	8
2.3.6 F. Profils et visibilité	8
2.3.7 G. Tableau de bord personnel	8
2.3.8 H. Administration	8
2.4 Confidentialité : exigence centrale du projet	9
2.5 Règles métier à faire respecter par le serveur	9
2.6 Modèle de données minimal	9
2.7 Points d'accès minimaux de l'API	10
2.8 Écrans React minimaux	12
2.9 Votre marge de créativité	12
2.10 Extensions valorisées	13
2.11 Scénario de validation	13
3 Exigences techniques transversales	14
3.1 TypeScript	14
3.2 Neon PostgreSQL	15
3.3 Prisma	15
3.4 Express	16
3.5 Authentification JWT	16
3.6 Axios	17
3.7 React	18
3.8 Git	18
4 Barème d'évaluation	18
4.1 Base de données et Prisma : 18 points	19
4.2 API Express : 26 points	19
4.3 Authentification, confidentialité et sécurité : 16 points	19
4.4 Interface React et Axios : 22 points	19
4.5 TypeScript, qualité du code et Git : 12 points	20
4.6 Documentation et remise : 6 points	20
4.7 Bonus : maximum 10 points	20
4.8 Pénalités	21
5 Livrables et modalités de remise	21
5.1 Ce que vous produisez	21
5.2 Ce que vous remettez sur Teams	21

6	Déroulement suggéré	22
7	Utilisation de l’intelligence artificielle	23
A	Gabarit du fichier de remise	23
B	Exemple de fichier de remise rempli	24
C	Démarrage rapide	25
D	Squelettes de référence	26
E	Pièges classiques	28
F	Liste de vérification avant la remise	28

1. Informations générales

1.1 Cadre de l'évaluation

Cet examen de mi-session prend la forme d'un hackathon de 88 heures. Vous travaillez en équipe de 2 à 6 personnes sur un projet unique, **MindHarbor**, et vous livrez une application web complète écrite **entièrement en TypeScript**, du serveur à l'interface.

Ouverture	Jeudi 13 août 2026, 8 h
Fermeture	Dimanche 16 août 2026, 23 h 59
Sujet	MindHarbor, imposé à toutes les équipes
Formation	Équipes de 2 à 6 personnes, un capitaine désigné
Remise	Un fichier <code>REMISE.md</code> déposé sur Teams, contenant le lien du dépôt GitHub public et les noms des membres
Note	100 points, plus un maximum de 10 points bonus
Présentation orale	Aucune. Le projet est corrigé à partir de votre dépôt.

Attention

Aucun retard ne sera accepté. **Le dernier commit poussé avant le dimanche 16 août 2026 à 23 h 59 fait foi**, et le fichier `REMISE.md` doit être déposé sur Teams avant cette même heure. Un commit poussé à 0 h 01 le 17 août ne sera pas corrigé.

Comme il n'y a pas de présentation orale, **votre dépôt parle à votre place**. Un projet qui ne démarre pas en suivant votre README est un projet qui ne sera pas vu. Consacrez les trois dernières heures à vérifier l'installation sur une machine vierge, pas à ajouter une fonctionnalité.

1.2 Objectifs pédagogiques

Au terme de cet examen, vous aurez démontré que vous savez :

- écrire une application full-stack **typée de bout en bout** avec TypeScript en mode strict ;
- modéliser un domaine métier en schéma **Prisma** (relations 1-N et N-N, enums, index, contraintes d'intégrité) ;
- provisionner et exploiter une base **PostgreSQL hébergée sur Neon** (migrations, seed, variables d'environnement) ;
- construire une **API REST avec Express** structurée en couches, validée et sécurisée ;
- protéger cette API avec **JWT** et un contrôle d'accès par rôle appliqué côté serveur ;
- consommer l'API depuis **React** au moyen d'une **instance Axios centralisée** et typée ;
- traiter des **données personnelles sensibles** avec la rigueur qu'elles exigent ;
- documenter votre travail assez clairement pour qu'un tiers l'installe et le comprenne sans vous.

1.3 Pile technologique imposée

Couche	Technologie imposée	Version
Langage	TypeScript , sur le serveur <i>et</i> sur le client	5.x
Exécution	Node.js, modules ESM	20 LTS
Serveur HTTP	Express, avec <code>@types/express</code>	4.18+
ORM	Prisma (<code>@prisma/client</code> et CLI <code>prisma</code>)	5.x ou 6.x
Base de données	PostgreSQL hébergée sur Neon	—
Interface	React, projet créé avec Vite, gabarit <code>react-ts</code>	18+
Client HTTP	Axios, instance configurée et typée	1.x
Authentification	<code>jsonwebtoken</code> et <code>bcryptjs</code> , avec leurs <code>@types</code>	—
Routage client	<code>react-router-dom</code>	6+

TypeScript est obligatoire, pas optionnel. Aucun fichier `.js` ou `.jsx` écrit à la main n'est accepté dans `src/` : votre code source est en `.ts` et `.tsx`. Les exigences précises de typage figurent à la section 3.1.

Librairies complémentaires encouragées. `zod` pour la validation, qui a l'avantage d'inférer vos types ; `cors`, `helmet`, `morgan`, `dotenv`, `express-rate-limit` ; `@tanstack/react-query` ou `zustand` ; une librairie d'interface (Tailwind, MUI, `shadcn/ui`) ; `recharts` ou `chart.js` pour les graphiques ; `date-fns` ou `dayjs` pour les dates ; `tsx` ou `ts-node-dev` pour l'exécution en développement.

Interdits. Tout ORM autre que Prisma (Sequelize, TypeORM, Mongoose) ; toute base de données autre que Neon PostgreSQL ; `fetch` et `XMLHttpRequest` pour appeler votre API depuis React ; tout framework frontal autre que React (Next.js est hors sujet : on attend une application React distincte d'une API Express) ; toute plateforme générant l'API automatiquement (Supabase auto-API, Firebase, Strapi, PocketBase).

1.4 Architecture attendue du dépôt

Deux applications distinctes dans un même dépôt, chacune avec son `package.json` et son `tsconfig.json` :

```
mindharbor/
+-- README.md
+-- REMISE.md           # fichier de remise, également depose sur Teams
+-- .gitignore          # node_modules, .env, dist
+-- docs/
|   +-- schema-bdd.png|md # diagramme ou description du schema
|   +-- api.md           # liste des endpoints
+-- server/
|   +-- .env.example     # variables SANS valeurs secretes
|   +-- package.json
|   +-- tsconfig.json    # "strict": true
|   +-- prisma/
|       +-- schema.prisma
|       +-- migrations/  # genere par prisma migrate dev
|       +-- seed.ts
|   +-- src/
|       +-- index.ts     # point d'entree : app.listen()
|       +-- app.ts       # configuration Express
|       +-- lib/prisma.ts # instance unique de PrismaClient
|       +-- routes/      # un fichier de routes par ressource
|       +-- controllers/ # lecture de req, ecriture de res
|       +-- services/    # logique metier et acces Prisma
|       +-- middlewares/ # auth.ts, validate.ts, error.ts
```

```

|      +-- schemas/           # schemas zod, source des types d'entree
|      +-- types/             # types partages, augmentation d'Express
|      +-- utils/
+-- client/
    +-- .env.example          # VITE_API_URL=...
    +-- package.json
    +-- tsconfig.json
    +-- src/
        +-- main.tsx
        +-- App.tsx
        +-- api/
            | +-- axios.ts      # instance Axios et intercepteurs
            | +-- journal.ts, groups.ts, resources.ts
        +-- types/             # types des entites renvoyees par l'API
        +-- context/AuthContext.tsx
        +-- components/
        +-- pages/
        +-- hooks/

```

Le respect de la séparation **routes** / **contrôleurs** / **services** fait partie de l'évaluation. Un fichier `index.ts` de 800 lignes contenant toute l'application est pénalisé, même s'il compile et fonctionne.

2. Le projet MindHarbor

2.1 Contexte

Selon Statistique Canada, un Canadien sur cinq vivra un problème de santé mentale au cours de sa vie. Pourtant, l'accès aux ressources demeure difficile : listes d'attente, coûts élevés, stigmatisation.

Une association de professionnels en santé mentale souhaite lancer **MindHarbor**, une plateforme qui combine le suivi personnel du bien-être, l'accès à des ressources adaptées et la participation à des groupes de soutien thématiques. L'objectif n'est pas de remplacer les professionnels, mais de démocratiser les outils de suivi et de créer des ponts vers l'aide appropriée.

Votre mandat : concevoir et développer cette plateforme, en traitant les données du journal personnel avec le niveau de confidentialité qu'elles exigent.

Ce sujet n'est pas un prétexte technique

Un Canadien sur cinq, cela veut dire quelqu'un dans votre équipe, dans votre classe, dans votre famille. Vous n'écrivez pas un questionnaire de tâches déguisé : vous écrivez une application que quelqu'un pourrait ouvrir un soir difficile.

Cela change des décisions très concrètes. Le ton de vos messages d'erreur. Ce qui s'affiche quand l'utilisateur n'a rien noté depuis dix jours : un reproche ou une main tendue. La visibilité des ressources d'urgence. Le choix de ne pas transformer la souffrance en score.

Prenez le sujet au sérieux, et prenez-le à cœur. Une équipe qui a manifestement réfléchi à l'expérience réelle d'une personne fragile se distingue immédiatement d'une équipe qui a coché des cases.

2.2 Acteurs du système

Visiteur	Consulte les ressources publiques et la liste des groupes, sans accéder au contenu des discussions. Peut s'inscrire et se connecter.
Utilisateur	Tient son journal, consulte ses tendances, parcourt les ressources et les met en favoris, rejoint des groupes, échange des messages privés selon ses paramètres de confidentialité.
Modérateur de groupe	Utilisateur avec des responsabilités supplémentaires <i>sur ses groupes uniquement</i> : approuve les demandes d'adhésion, modère les publications, gère les membres.
Administrateur	Consulte des statistiques anonymisées, traite la file de signalements, publie des ressources officielles, valide les comptes professionnels si vous implémentez cette extension.

2.3 Fonctionnalités du noyau obligatoire

Noyau et extensions

Le projet se divise en un **noyau obligatoire** (cette section) et des **extensions valorisées** (section 2.10). **Le noyau doit être complet et fonctionnel avant que vous touchiez à une seule extension.** Une équipe qui livre un noyau impeccable obtient une meilleure note qu'une équipe qui livre six fonctionnalités à moitié terminées. La correction commence toujours par le noyau, et une fonctionnalité qui plante ne rapporte rien.

2.3.1 A. Journal de bien-être

L'utilisateur enregistre une entrée quotidienne comportant :

- l'humeur générale, sur une échelle de 1 à 5 ;
- le niveau d'énergie, de 1 à 5 ;
- la qualité du sommeil de la nuit précédente, de 1 à 5 ;
- le niveau d'anxiété ou de stress, de 1 à 5 ;
- les activités réalisées, choisies dans un catalogue (exercice, social, travail, méditation, loisirs, etc.) : il s'agit d'une **relation N-N** entre l'entrée et les activités ;
- les événements marquants de la journée, en texte libre ;
- une gratitude du jour, facultative.

Attention

Une seule entrée par utilisateur et par jour. Cette règle doit être imposée *en base de données* par une contrainte `@@unique([userId, date])`, et pas seulement vérifiée en TypeScript. L'entrée reste modifiable jusqu'à minuit le jour même ; passé ce délai, le serveur refuse toute modification avec un code 403.

Comportement attendu

Chaque soir, Thomas ouvre MindHarbor et remplit son journal : humeur 3/5, énergie 2/5, sommeil 4/5, anxiété 4/5 après une journée stressante au travail. Il coche l'activité « marche » et écrit dans les événements marquants : « Conflit avec un collègue ». En gratitude, il note : « Bon repas avec ma sœur ce midi ».

2.3.2 B. Analyse et tendances

Le serveur, et non le client, calcule et renvoie :

- l'évolution de l'humeur, de l'énergie, du sommeil et de l'anxiété sur 7, 30 et 90 jours ;
- les moyennes par jour de la semaine, afin de révéler les « lundis difficiles » ;
- **au moins une corrélation activité–indicateur** : la moyenne d'un indicateur les jours *avec* une activité donnée, comparée aux jours *sans* cette activité ;
- au moins deux observations générées automatiquement à partir de ces chiffres, avec un seuil minimal de données avant de conclure (par exemple au moins cinq jours dans chacun des deux groupes comparés).

Comportement attendu

Après trois semaines d'utilisation, Sophie consulte ses tendances. Elle voit un graphique de son humeur au fil du temps, avec une légère tendance à la hausse, et un encadré lui indique : « Les jours où vous notez l'activité méditation, votre anxiété est en moyenne 30 % plus basse. »

Attention

Les agrégats viennent de Prisma (`groupBy`, `_avg`, `_count`), **jamais** d'un `findMany()` qui rapatrie toutes les entrées suivi d'une boucle. Les graphiques côté React s'appuient sur `recharts` ou `chart.js`.

2.3.3 C. Ressources et exercices

La plateforme propose une bibliothèque de ressources classées : articles éducatifs, exercices guidés (respiration, méditation, écriture), fiches pratiques, liens vers des contenus audio ou vidéo externes.

- Chaque ressource porte une catégorie thématique (anxiété, sommeil, relations, travail, deuil), un type, une durée et un niveau.
- Recherche par mot-clé et filtres par catégorie, type et durée, avec pagination.
- Système de **favoris** : relation N-N entre l'utilisateur et les ressources.
- **Suggestion contextuelle** : si la dernière entrée de journal indique une anxiété supérieure ou égale à 4, la page d'accueil propose une ressource de la catégorie « anxiété ».

Comportement attendu

Marc ressent de l'anxiété. Il ouvre la bibliothèque, filtre par « anxiété » et trouve un exercice de respiration 4-7-8 de cinq minutes. Il le réalise et l'ajoute à ses favoris. Le lendemain, comme son journal indique une anxiété élevée, la plateforme lui suggère cet exercice dès la page d'accueil.

2.3.4 D. Groupes de soutien

Les utilisateurs rejoignent des groupes thématiques pour échanger avec des personnes vivant des situations similaires.

- Un groupe possède une thématique, une description, des règles de communauté affichées et une visibilité `PUBLIC` ou `PRIVE`.
- Un groupe privé exige l'approbation d'un modérateur : modélisez une demande d'adhésion avec un statut (`EN_ATTENTE`, `ACCEPTEE`, `REFUSEE`).

- Le créateur d'un groupe en devient automatiquement modérateur, et un groupe doit toujours conserver au moins un modérateur actif.
- Fil de discussion : publications et commentaires, avec pagination.
- Liste des membres, filtrée selon leurs paramètres de confidentialité.
- Signalement de contenu, avec catégories.

Comportement attendu

Julie souffre d'anxiété sociale. Elle trouve le groupe « Apprivoiser l'anxiété sociale » (45 membres, groupe privé) et demande à le rejoindre en écrivant une courte présentation. Le modérateur accepte. Julie lit les discussions récentes, publie son premier message et reçoit deux réponses de soutien.

2.3.5 E. Messagerie privée et confidentialité des échanges

Chaque utilisateur choisit qui peut le contacter, parmi trois niveaux :

PERSONNE	Aucun message privé entrant.
MEMBRES_DE_MES_GROUPEs	Seuls les membres d'un groupe partagé peuvent écrire.
TOUT_LE_MONDE	Tout membre inscrit peut écrire.

Le serveur applique ce paramètre. Tenter d'écrire à quelqu'un qui ne l'accepte pas renvoie 403. Prévoyez également le blocage d'un utilisateur et un indicateur de messages non lus.

2.3.6 F. Profils et visibilité

L'utilisateur contrôle son identité affichée : pseudonyme ou nom réel, avatar, courte biographie, et un niveau de visibilité du profil parmi PUBLIC, GROUPEs_SEULEMENT et PRIVE. Ces règles doivent être appliquées dans les `select` Prisma des points d'accès publics, et non masquées uniquement dans l'interface React.

Comportement attendu

Thomas utilise le pseudonyme « Tom_mtl » et garde son profil en mode « groupes seulement ». Quand il publie dans le groupe burnout, les autres membres voient son pseudonyme et son avatar, mais ne peuvent ni consulter ses autres groupes ni le contacter directement, car il a désactivé la messagerie.

2.3.7 G. Tableau de bord personnel

Une vue centralisée qui rassemble : le résumé de la semaine (humeur moyenne et tendance), la suggestion du jour, l'état du journal (rempli ou non aujourd'hui), les prochains événements des groupes le cas échéant, les notifications récentes et un accès rapide à la saisie du journal.

2.3.8 H. Administration

- Statistiques d'utilisation **anonymisées** : nombre d'utilisateurs actifs, nombre d'entrées de journal, groupes les plus actifs. Jamais le contenu d'un journal.
- File de signalements triée par priorité, avec traitement (contenu masqué, avertissement, rejet du signalement).
- Création et publication de ressources officielles.
- Suspension d'un compte : un compte suspendu ne peut plus se connecter et reçoit 403.

2.4 Confidentialité : exigence centrale du projet

Ce sujet manipule des données de santé. Votre conception doit refléter cette réalité, et ces points seront testés explicitement pendant la correction.

Attention

Le contenu d'un journal n'est accessible qu'à son auteur. Une requête GET `/api/v1/journal/:date` portant sur l'entrée d'un autre utilisateur doit renvoyer 403 ou 404, **y compris pour un compte administrateur**. Le correcteur tentera cet appel depuis un second compte.

- Les statistiques destinées à l'administration sont agrégées et anonymisées.
- L'utilisateur peut **exporter ses données** au format JSON ou CSV.
- L'utilisateur peut **supprimer son compte**, et vous documentez les conséquences retenues : suppression en cascade ou anonymisation des publications déjà écrites. Les deux choix sont acceptables ; l'absence de choix documenté ne l'est pas.
- Les contenus signalés comme **INQUIETANT** (risque suicidaire) remontent **en tête** de la file d'administration et déclenchent une alerte visible.
- Les ressources d'aide d'urgence, avec un numéro de ligne de crise, sont accessibles **depuis toutes les pages** de l'application.
- Le ton de l'interface doit rester bienveillant et non jugeant. Aucune formulation culpabilisante du type « Vous avez encore oublié votre journal ».

2.5 Règles métier à faire respecter par le serveur

Journal	Une entrée par utilisateur et par jour, imposée par <code>@unique([userId, date])</code> . Modification permise jusqu'à minuit le jour même, refusée ensuite.
Tendances	Aucune conclusion affichée sous le seuil minimal de données que vous fixez et documentez.
Groupes	Un utilisateur peut appartenir à plusieurs groupes. Un groupe privé exige l'approbation d'un modérateur. Le créateur devient modérateur. Un groupe conserve au moins un modérateur actif : le dernier modérateur ne peut pas quitter le groupe sans en désigner un autre.
Publications	Seul l'auteur ou un modérateur du groupe concerné peut supprimer une publication. Un non-membre ne voit pas le contenu d'un groupe privé.
Messagerie	Le niveau de contact du destinataire est vérifié côté serveur à chaque envoi. Un utilisateur bloqué ne peut plus écrire.
Signalements	Catégories obligatoires : INAPPROPRIÉ , SPAM , INQUIETANT . Un même utilisateur ne signale pas deux fois le même contenu.

2.6 Modèle de données minimal

Votre schéma Prisma doit comporter au minimum les modèles suivants :

Modèle	Rôle
User	Compte, rôle, paramètres de confidentialité et de contact
JournalEntry	Entrée quotidienne, contrainte d'unicité sur (utilisateur, date)
Activity	Catalogue d'activités
JournalActivity	Table de jonction N-N entre entrée et activités
Resource	Article, exercice ou fiche pratique
Favorite	Relation N-N entre utilisateur et ressource
Group	Groupe de soutien, visibilité publique ou privée
GroupMember	Appartenance et rôle dans le groupe, statut de la demande
Post	Publication dans un groupe
Comment	Commentaire sur une publication
Message	Message privé entre deux utilisateurs
Report	Signalement de contenu, avec catégorie et statut
RefreshToken	Jetons de rafraîchissement, pour l'invalidation serveur

Exigences de modélisation, reprises dans le barème :

- au moins une relation **1-N** et une relation **N-N** correctement modélisées ;
- au moins **deux enums** : visibilité du profil, niveau de contact, rôle, statut de demande ou catégorie de signalement ;
- `@id`, `createdAt` et `updatedAt` sur tous les modèles pertinents ;
- au moins **trois index** (`@@index`) ou contraintes d'unicité composées, **justifiés dans le README** par une requête réelle de votre application ;
- `onDelete` explicite sur chaque relation : réfléchissez à ce qui doit arriver aux publications d'un membre qui supprime son compte ;
- au moins une **transaction prisma** `prisma.$transaction` là où l'atomicité est nécessaire, par exemple l'acceptation d'une demande d'adhésion qui crée l'appartenance et clôt la demande ;
- au moins une **agrégation Prisma** pour les tendances et les statistiques.

2.7 Points d'accès minimaux de l'API

Toutes les routes sont préfixées par `/api/v1`. Les listes acceptent `?page=`, `?limit=`, `?sort=` et `?order=`.

Authentification

Méthode	Route	Accès
POST	<code>/auth/register</code>	Public
POST	<code>/auth/login</code>	Public
POST	<code>/auth/refresh</code>	Public, jeton valide
POST	<code>/auth/logout</code>	Authentifié
GET	<code>/auth/me</code>	Authentifié

Journal et tendances

Méthode	Route	Accès
GET	/journal (liste paginée de mes entrées)	Auteur seulement
POST	/journal (entrée du jour)	Auteur seulement
GET	/journal/:date	Auteur seulement
PATCH	/journal/:date (jusqu'à minuit)	Auteur seulement
GET	/journal/stats?range=30d	Auteur seulement
GET	/journal/insights	Auteur seulement

Ressources et favoris

Méthode	Route	Accès
GET	/activities	Public
GET	/resources (recherche, filtres, pagination)	Public
GET	/resources/:id	Public
POST	/resources	Administrateur
POST	/resources/:id/favorite	Authentifié
DELETE	/resources/:id/favorite	Authentifié
GET	/me/favorites	Authentifié
GET	/me/suggestions	Authentifié

Groupes de soutien

Méthode	Route	Accès
GET	/groups (liste, recherche)	Public
POST	/groups	Authentifié
GET	/groups/:id (aperçu ou contenu selon l'adhésion)	Public ou membre
POST	/groups/:id/join	Authentifié
GET	/groups/:id/requests	Modérateur
PATCH	/groups/:id/requests/:requestId	Modérateur
DELETE	/groups/:id/members/:userId	Modérateur
GET	/groups/:id/posts	Membre
POST	/groups/:id/posts	Membre
DELETE	/posts/:id	Auteur ou modérateur
POST	/posts/:id/comments	Membre

Messagerie

Méthode	Route	Accès
GET	/messages (liste des conversations)	Authentifié
GET	/messages/:userId (fil)	Authentifié
POST	/messages/:userId	Selon le niveau de contact
POST	/users/:id/block	Authentifié

Profil et confidentialité

Méthode	Route	Accès
GET	/users/:id (profil, filtré par visibilité)	Public
PATCH	/me (profil)	Authentifié
PATCH	/me/privacy	Authentifié
GET	/me/export	Authentifié
DELETE	/me	Authentifié

Signalements et administration

Méthode	Route	Accès
POST	/reports	Authentifié
GET	/admin/reports (triés, urgents en tête)	Administrateur
PATCH	/admin/reports/:id	Administrateur
GET	/admin/stats (anonymisées)	Administrateur
PATCH	/admin/users/:id/suspend	Administrateur

Vous pouvez ajouter des routes, mais celles-ci constituent le minimum attendu.

2.8 Écrans React minimaux

1. **Accueil** : présentation, aperçu des groupes publics, accès aux ressources, ressources d'urgence visibles.
2. **Inscription** et **Connexion** : formulaires validés côté client et côté serveur.
3. **Tableau de bord** : résumé de la semaine, suggestion du jour, accès rapide au journal.
4. **Journal** : formulaire de saisie du jour et historique paginé.
5. **Mes tendances** : graphiques et observations automatiques.
6. **Ressources** : bibliothèque avec recherche, filtres et favoris.
7. **Groupes** : liste, recherche, mes groupes.
8. **Détail d'un groupe** : fil de discussion, membres, demandes en attente pour un modérateur.
9. **Profil et confidentialité** : identité affichée, visibilité, niveau de contact, export et suppression.
10. **Messagerie** : liste des conversations et fil de discussion.
11. **Administration** : statistiques, file de signalements, gestion des ressources.

2.9 Votre marge de créativité

Le cahier des charges ci-dessus fixe un **plancher**, pas un plafond. Deux équipes peuvent le respecter intégralement et livrer deux produits qui n'ont rien à voir. Ce qui vous appartient :

- **L'identité visuelle et le ton.** Palette, typographie, rythme des pages, formulation de chaque message. Une interface apaisante n'est pas un accident : c'est une décision.
- **Les dimensions suivies.** Le journal impose quatre indicateurs ; rien ne vous empêche d'en proposer d'autres, si vous justifiez le choix.
- **La façon de présenter les tendances.** Un graphique n'est pas la seule réponse. Une phrase juste vaut parfois mieux qu'un histogramme.
- **Les micro-attentions.** Ce qui se passe le premier jour, quand il n'y a encore aucune donnée. Ce qui s'affiche après une semaine d'absence. La façon dont on quitte l'application.
- **Une fonctionnalité de votre invention**, cohérente avec la mission de la plateforme et pleinement intégrée au reste. Elle vaut jusqu'à **4 points bonus** (section 4.7).

Le test du « pire jour »

Avant de livrer, faites cet exercice ensemble. Imaginez une personne qui ouvre votre application le soir de sa pire journée du mois. Parcourez votre interface avec ses yeux, écran par écran. Est-ce que quelque chose la culpabilise ? Est-ce qu'elle trouve de l'aide en moins de deux clics ? Est-ce que le vocabulaire employé la respecte ? Corrigez ce qui accroche. C'est souvent une heure de travail, et cela change tout le projet.

2.10 Extensions valorisées

Ces extensions ne sont pas obligatoires. Elles rapportent des points bonus, cumulables avec les autres bonus jusqu'à un maximum de **10 points** pour l'ensemble du projet.

Extension	Points	Description
Défis collectifs	+3	Objectif de groupe sur une durée définie, suivi de progression individuel, statistiques agrégées.
Événements de groupe	+3	Création par un modérateur, inscriptions, rappels avant l'événement.
Badges de régularité	+2	7, 30 et 100 jours de journal, affichables ou masquables par l'utilisateur.
Professionnels vérifiés	+4	Comptes distincts avec qualifications, validés par un administrateur, autorisés à publier des ressources officielles.
Messagerie temps réel	+4	Socket.io pour les messages privés et les notifications.
Journal hors ligne	+4	PWA avec file de synchronisation lors du retour en ligne.
Export PDF mis en forme	+2	Journal et statistiques personnelles, au-delà du simple JSON ou CSV.

2.11 Scénario de validation

Il n'y a pas de présentation orale : **c'est le correcteur qui exécutera ce scénario sur votre application**, à partir de votre README et de votre seed. Déroulez-le vous-même de bout en bout avant la remise, sur une installation propre.

1. Léa s'inscrit avec un pseudonyme et règle son profil en mode privé, messagerie désactivée.
2. Sur le compte du seed disposant de trente jours de données, la page « Mes tendances » affiche le graphique d'évolution et au moins une observation calculée automatiquement.
3. Léa saisit son entrée du jour. Une seconde saisie pour la même date est refusée par le serveur, et l'interface affiche un message clair.
4. Marc demande à rejoindre un groupe privé. Le modérateur voit la demande et l'accepte. Marc publie un message, un autre membre le commente.
5. Un membre signale une publication comme INQUIETANT. L'administrateur ouvre sa file : le signalement apparaît **en tête**, avec une alerte visible.
6. Depuis un second compte, un appel à l'entrée de journal de Léa est refusé par l'API.
7. Léa exporte ses données et le fichier se télécharge.
8. Les ressources d'aide d'urgence sont atteignables depuis n'importe quelle page.

Documentez dans votre README tout écart entre ce scénario et votre application. Un écart déclaré coûte moins cher qu'un écart découvert.

3. Exigences techniques transversales

Ces exigences sont vérifiées à la correction et représentent la majorité des points techniques.

3.1 TypeScript

L'ensemble du projet est écrit en TypeScript, avec le mode strict activé des deux côtés.

Listing 1 – server/tsconfig.json – configuration minimale attendue

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true, // obligatoire
    "noImplicitAny": true,
    "noUnusedLocals": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src/**/*.ts", "prisma/**/*.ts"]
}
```

Exigences de typage.

- "strict": true dans les deux tsconfig.json. Le désactiver, ou le contourner par un // @ts-nocheck, annule les points correspondants.
- **Aucun any implicite.** Un any explicite reste tolérable s'il est rare et accompagné d'un commentaire justifiant son emploi. Devant une valeur de type inconnu, préférez unknown suivi d'un affinage.
- **La commande npx tsc -noEmit doit passer sans erreur** dans server/ comme dans client/. C'est le premier test exécuté à la correction.
- **Réutilisez les types générés par Prisma** plutôt que de redéclarer vos entités à la main : import type { User, JournalEntry } from '@prisma/client'.
- **Dérivez vos types d'entrée de vos schémas zod** avec z.infer, pour qu'une seule source décrive à la fois la validation et le type.
- **Typez req.user** par augmentation de l'interface Request d'Express, et non par un as any à chaque contrôleur.
- **Typez les réponses Axios** au moyen des génériques, par exemple api.get<JournalEntry[]>(), pour que la donnée arrive typée dans le composant.
- Les props de chaque composant React sont typées. Pas de props: any.

Listing 2 – server/src/types/express.d.ts – augmentation propre de Request

```
import type { Role } from '@prisma/client';

declare global {
  namespace Express {
    interface Request {
      user?: { id: string; role: Role };
    }
  }
}
```

```
export {};
```

Partage des types entre le serveur et le client

Le client a besoin des mêmes formes de données que le serveur. Deux approches sont acceptées : un dossier `shared/` importé par les deux applications, ou un dossier `client/src/types/` qui redéclare les interfaces exposées par l'API. La première est plus élégante, la seconde est plus simple à configurer en 88 heures. **Choisissez et justifiez votre choix dans le README** ; ce qui est sanctionné, c'est l'absence totale de types côté client.

3.2 Neon PostgreSQL

- Chaque équipe crée **un projet Neon** sur le plan gratuit et y connecte Prisma.
- La chaîne de connexion vit dans `server/.env` sous `DATABASE_URL`, avec `?sslmode=require`.
- Le fichier `.env` **ne doit jamais être commité**. Un `.env.example` documenté le remplace dans le dépôt.
- Le dossier `prisma/migrations/`, généré par `npx prisma migrate dev`, doit être présent dans le dépôt. Utiliser `prisma db push` sans historique de migration coûte des points.
- Une branche Neon de développement, ou une seconde base, vous permet de tester le seed sans détruire les données de démonstration. C'est valorisé.

Listing 3 – `server/.env.example`

```
DATABASE_URL="postgresql://<user>:<password>@<host>.neon.tech/<db>?sslmode=require"
JWT_ACCESS_SECRET="a-replacer"
JWT_REFRESH_SECRET="a-replacer"
ACCESS_TOKEN_TTL="15m"
REFRESH_TOKEN_TTL="7d"
PORT=3000
CLIENT_URL="http://localhost:5173"
```

Mise en veille

Neon met les bases inactives en veille. Le premier appel après une pause peut prendre une à deux secondes : prévoyez-le dans vos états de chargement. **Votre base doit rester accessible et peuplée après la remise**, puisque la correction aura lieu plus tard.

3.3 Prisma

Une seule instance de `PrismaClient` pour toute l'application :

Listing 4 – `server/src/lib/prisma.ts`

```
import { PrismaClient } from '@prisma/client';

export const prisma = new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['query', 'error'] : ['error'],
});
```

Attention

Instancier `new PrismaClient()` dans plusieurs fichiers épuise le pool de connexions de Neon et provoque des erreurs intermittentes difficiles à diagnostiquer. C'est une erreur classique, et elle est pénalisée.

Seed. Le script `prisma/seed.ts`, lancé par `npm run seed`, doit créer un jeu de données crédible : **au moins 10 utilisateurs**, dont un administrateur dont les identifiants figurent dans le README, **au moins un compte avec 30 jours d'entrées de journal** pour que les tendances soient démontrables, plusieurs groupes dont un privé, une vingtaine de ressources réparties en catégories, et assez de publications pour que la pagination soit visible. Le seed doit pouvoir être relancé sans faire exploser les contraintes d'unicité.

3.4 Express

- `express.json()`, cors configuré sur l'origine du client, `helmet`, `morgan`.
- Routes montées sous un préfixe versionné `/api/v1`.
- Un `express.Router()` par ressource : aucune route déclarée directement sur `app`.
- Un **middleware d'erreurs centralisé** monté en dernier. Aucun contrôleur ne construit sa propre réponse d'erreur à la main.
- Un gestionnaire 404 pour les routes inconnues.
- Validation de **toutes** les entrées (corps, paramètres, requête) avec `zod`, dans un middleware réutilisable.
- Codes HTTP appropriés : 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500.
- `express-rate-limit` sur `/auth/login` et `/auth/register` : fortement recommandé.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Les donnees envoyees sont invalides.",
    "details": [ { "field": "email", "message": "Format de courriel invalide" } ]
  }
}
```

```
{
  "data": [ ... ],
  "meta": { "page": 2, "limit": 20, "total": 137, "totalPages": 7 }
}
```

Toutes les listes acceptent la pagination, au moins un tri et au moins deux filtres métier, avec un `limit` **plafonné côté serveur** (100 par exemple) pour qu'un client ne puisse pas réclamer dix mille lignes.

3.5 Authentification JWT

Format de liste paginée uniforme sur toute l'API.

- **POST** `/auth/register` : validation (courriel valide, mot de passe d'au moins 8 caractères comprenant une majuscule et un chiffre), hachage `bcrypt` avec au moins 10 tours, doublon de courriel traité en 409.
- **POST** `/auth/login` : renvoie un **jeton d'accès** de courte durée (15 minutes) et un **jeton de rafraîchissement** (7 jours).
- **POST** `/auth/refresh` : renouvelle le jeton d'accès.
- **POST** `/auth/logout` : invalide le jeton de rafraîchissement **côté serveur**, via la table `RefreshToken`. Vider le `localStorage` du navigateur ne compte pas comme une déconnexion.
- Le hachage du mot de passe ne doit **jamais** apparaître dans une réponse de l'API. Utilisez un `select` Prisma explicite ou une fonction de sérialisation typée.

- Middlewares `requireAuth` et `requireRole`. Masquer un bouton dans React n'est pas une protection.
- **Vérification de la propriété des ressources** : un utilisateur ne modifie et ne supprime que ses propres enregistrements. Une route qui laisse n'importe qui supprimer la publication d'un autre est une faute grave.

3.6 Axios

Tous les appels réseau passent par **une seule instance Axios**, exportée depuis le fichier `client/src/api/axios.ts`. Cette instance doit lire son `baseUrl` depuis `import.meta.env.VITE_API_URL`, attacher le jeton d'accès par un intercepteur de requête, intercepter les réponses 401 pour tenter un rafraîchissement puis **rejouer la requête d'origine**, et déconnecter proprement si le rafraîchissement échoue.

Listing 5 – `client/src/api/axios.ts`

```
import axios, { AxiosError, type InternalAxiosRequestConfig } from 'axios';

type RetriableConfig = InternalAxiosRequestConfig & { _retry?: boolean };

export const api = axios.create({
  baseUrl: import.meta.env.VITE_API_URL, // ex. http://localhost:3000/api/v1
  timeout: 10000,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('accessToken');
  if (token) config.headers.Authorization = 'Bearer ' + token;
  return config;
});

let refreshing: Promise<{ data: { accessToken: string } }> | null = null;

api.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const original = error.config as RetriableConfig | undefined;
    if (error.response?.status === 401 && original && !original._retry) {
      original._retry = true;
      try {
        refreshing ??= api.post('/auth/refresh', {
          refreshToken: localStorage.getItem('refreshToken'),
        });
        const { data } = await refreshing;
        refreshing = null;
        localStorage.setItem('accessToken', data.accessToken);
        return api(original); // on rejoue la requete d'origine
      } catch {
        refreshing = null;
        localStorage.clear();
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);
```

Vous devez également regrouper les appels par ressource dans `client/src/api/` (`journal.ts`, `groups.ts`, `resources.ts`), avec un type de retour explicite pour chaque fonction : **aucun `api.get(...)` directement dans un composant TSX**. L'annulation des requêtes des composants démontés, au moyen d'un `AbortController`, est valorisée.

Listing 6 – `client/src/api/journal.ts` – appels typés

```
import { api } from '../axios';
import type { JournalEntry, Paginated } from '../types';
```

```
export async function getJournal(page = 1): Promise<Paginated<JournalEntry>> {  
  const { data } = await api.get<Paginated<JournalEntry>>('/journal', {  
    params: { page, limit: 20 },  
  });  
  return data;  
}
```

3.7 React

- Projet créé avec **Vite**, gabarit `react-ts`.
- Routage avec `react-router-dom`, incluant un composant `<ProtectedRoute>` et un composant `<AdminRoute>`.
- État d'authentification global via l'API Context, Zustand ou React Query, avec un type de contexte explicite. Pas de transmission de propriétés sur cinq niveaux.
- Composants réutilisables : au minimum un composant de pagination, un champ de formulaire, un indicateur de chargement et un affichage d'erreur.
- Validation côté client **en plus** de la validation serveur, jamais à sa place.
- Interface **adaptive**, testée à 375 pixels de large au minimum.
- Trois états traités pour chaque appel : chargement, erreur et **état vide** explicite, rédigé avec soin.
- Retour visuel systématique : bouton désactivé pendant l'envoi, message de succès ou d'erreur.

3.8 Git

- Le dépôt GitHub est **public** : le correcteur doit pouvoir le cloner sans invitation.
- **Au moins 30 commits significatifs**, répartis entre tous les membres. La commande `git log -author` sera consultée.
- Messages descriptifs : `feat: endpoint de creation d'entree de journal`, et non `update`, `fix` ou `test2`.
- `node_modules/`, `.env` et `dist/` dans le `.gitignore` dès le premier commit.

Attention

Un dépôt de trois commits, tous du même auteur, pour une équipe de quatre personnes sera traité comme un problème de contribution et entraînera un ajustement des notes individuelles. Comme il n'y a pas de présentation orale, **l'historique Git est la principale trace de votre participation.**

4. Barème d'évaluation

Le projet est noté sur **100 points**, auxquels s'ajoutent jusqu'à **10 points bonus**. Il n'y a pas de présentation orale : chaque critère est vérifié directement dans votre dépôt et dans l'application en fonctionnement.

4.1 Base de données et Prisma : 18 points

Critère	Points
Schéma pertinent : au moins 13 modèles attendus, relations 1-N et N-N correctes, enums	6
Contraintes et intégrité : <code>@@unique</code> sur (utilisateur, date), <code>onDelete</code> explicites	3
Index justifiés dans le README par une requête réelle	3
Migrations Prisma versionnées dans le dépôt	2
Seed complet : 10 utilisateurs, un compte avec 30 jours de journal, groupes et ressources	4

4.2 API Express : 26 points

Critère	Points
Couverture fonctionnelle du noyau : journal, tendances, ressources, groupes, messagerie, administration	9
Architecture en couches : routeurs, contrôleurs, services, middlewares	5
Conventions REST : ressources, verbes, codes HTTP corrects	4
Validation serveur systématique du corps, des paramètres et de la requête	3
Middleware d'erreurs centralisé et format d'erreur uniforme	2
Pagination, tri et filtres sur toutes les listes, avec <code>limit</code> plafonné	3

4.3 Authentification, confidentialité et sécurité : 16 points

Critère	Points
Inscription, connexion, rafraîchissement et déconnexion fonctionnels, hachage bcrypt	4
Jeton d'accès et jeton de rafraîchissement, avec invalidation serveur	3
Middlewares <code>requireAuth</code> et <code>requireRole</code> , permissions vérifiées côté serveur	3
Journal strictement privé et paramètres de confidentialité appliqués par le serveur	4
Vérification de la propriété des ressources : aucun accès horizontal possible	2

4.4 Interface React et Axios : 22 points

Critère	Points
Instance Axios unique et typée, <code>baseUrl</code> par variable d'environnement, intercepteur de requête	4
Intercepteur de réponse : rafraîchissement sur 401 et rejeu de la requête	3
Couche <code>api/</code> séparée des composants, aucune URL en dur dans le TSX	3
Routes protégées et état d'authentification global	2
Couverture des écrans minimaux, dont les graphiques de tendances	5
États de chargement, d'erreur et vides traités partout, retour visuel	3
Interface adaptative, cohérence visuelle et ton bienveillant	2

4.5 TypeScript, qualité du code et Git : 12 points

Critère	Points
<code>strict: true</code> des deux côtés et <code>tsc -noEmit</code> sans erreur	4
Typage réel : types Prisma réutilisés, <code>z.infer</code> , <code>req.user</code> typé, réponses Axios génériques, absence de <code>any</code> de complaisance	3
Organisation, nommage cohérent, absence de duplication flagrante	2
Aucun secret commité, <code>.gitignore</code> correct, <code>.env.example</code> fourni	1
Historique Git : au moins 30 commits significatifs répartis dans l'équipe	2

4.6 Documentation et remise : 6 points

Critère	Points
README : installation, variables d'environnement, commandes, comptes de démonstration	3
Documentation des points d'accès : <code>docs/api.md</code> , collection Postman ou Swagger	1
Diagramme ou description du schéma, et justification des choix techniques	1
Fichier REMISE.md complet et conforme au gabarit, déposé sur Teams dans les temps	1

Total : 18 + 26 + 16 + 22 + 12 + 6 = 100 points.

4.7 Bonus : maximum 10 points

Cumulables jusqu'à 10 points au total :

Bonus	Points
Les extensions listées à la section 2.10	voir tableau
Créativité : une fonctionnalité de votre invention, pertinente et bien intégrée, ou un soin remarquable apporté à l'expérience des personnes fragiles	jusqu'à +4
Tests automatisés (Vitest, Jest ou Supertest) sur des cas réels	+4
Déploiement en ligne fonctionnel (Render, Railway, Vercel)	+4
Types partagés entre serveur et client dans un dossier commun	+2
Documentation Swagger générée	+2
Intégration continue GitHub Actions exécutant <code>tsc -noEmit</code>	+2
Accessibilité soignée : navigation au clavier, contrastes, attributs ARIA	+2

4.8 Pénalités

Situation	Pénalité
Le projet ne démarre pas en suivant le README sur une machine vierge	−10
Le dépôt GitHub n'est pas public au moment de la correction	−10
Code écrit en JavaScript au lieu de TypeScript, ou <code>strict</code> désactivé	−10
Mot de passe stocké en clair, ou hachage renvoyé par l'API	−8
Entrée de journal accessible par un autre utilisateur	−8
Secret commité : <code>.env</code> , chaîne Neon, clé JWT	−5
<code>fetch</code> utilisé à la place d'Axios pour appeler l'API	−5
Permission vérifiée uniquement côté React	−5
<code>node_modules/</code> commité	−3
Contribution très déséquilibrée dans l'historique Git	Note individuelle ajustée

5. Livrables et modalités de remise

5.1 Ce que vous produisez

1. Un **dépôt GitHub public** contenant les dossiers `server/` et `client/`. Vérifiez la visibilité en ouvrant l'URL dans une fenêtre de navigation privée : si vous voyez le code sans être connecté, c'est bon.
2. Un **fichier README.md** à la racine, comprenant :
 - le nom de l'équipe et le nom complet de chaque membre ;
 - l'installation pas à pas : clonage, `npm install`, variables d'environnement, `prisma migrate deploy`, `npm run seed`, démarrage des deux applications ;
 - les **comptes de démonstration** avec leurs mots de passe : au minimum un administrateur, un modérateur de groupe et deux utilisateurs, dont celui qui possède 30 jours de journal ;
 - le schéma de base de données, sous forme de diagramme ou de description des modèles et des relations ;
 - la liste des points d'accès, avec méthode, chemin, accès requis et description ;
 - vos **choix techniques justifiés** : pourquoi ces index, cette stratégie de jetons, cette approche de partage des types, cette politique de suppression de compte ;
 - les fonctionnalités **non terminées**, déclarées honnêtement.
3. Les fichiers `server/.env.example` et `client/.env.example`.
4. Une **base Neon fonctionnelle et peuplée**, qui doit le rester après la remise.

5.2 Ce que vous remettez sur Teams

Le capitaine dépose sur Teams, dans le devoir prévu à cet effet, **un seul fichier nommé REMISE.md** avant le **dimanche 16 août 2026, 23 h 59**. Le même fichier est aussi commité à la racine de votre dépôt.

Le gabarit à remplir est fourni avec cet énoncé sous le nom `REMISE-template.md`. Il est reproduit à l'annexe A, suivi d'un exemple rempli à l'annexe B.

Attention

Trois erreurs rendent une remise incorrigible, et elles arrivent chaque session : un dépôt resté **privé**, un lien pointant vers une **branche locale jamais poussée**, et un **REMISE.md sans les noms complets** des membres. Vérifiez ces trois points avant de déposer.

6. Déroulement suggéré

Ce calendrier est indicatif, mais il reflète un découpage réaliste des 88 heures.

Jour	Plage	Objectif
Jeu. 13 août	8 h–11 h	Lecture de l'énoncé, modélisation du schéma Prisma sur papier, découpage des tâches, maquettes des écrans, décisions de ton et d'identité visuelle.
	11 h–13 h	Création du projet Neon, initialisation des deux applications en TypeScript, <code>tsconfig</code> en mode strict, <code>schema.prisma</code> , première migration, dépôt GitHub public.
	13 h–18 h	Authentification complète : inscription, connexion, rafraîchissement, déconnexion, middlewares <code>requireAuth</code> et <code>requireRole</code> , augmentation typée de <code>Request</code> .
	18 h–22 h	Seed, catalogue d'activités, premières routes du journal.
Ven. 14 août	8 h–13 h	Journal complet, contrainte d'unicité, règle de modification jusqu'à minuit, endpoints de tendances avec agrégations Prisma.
	13 h–18 h	Mise en place de React et Vite, instance Axios typée et intercepteurs, contexte d'authentification, écrans de connexion et d'inscription.
	18 h–22 h	Écrans du journal et des tendances branchés sur l'API, premiers graphiques.
Sam. 15 août	8 h–14 h	Groupes : création, adhésion, approbation, publications et commentaires. Ressources et favoris.
	14 h–20 h	Messagerie et paramètres de confidentialité, tableau de bord, administration et signalements.
	20 h–22 h	Gel des nouvelles fonctionnalités. Intégration et correction de bogues uniquement.
Dim. 16 août	8 h–13 h	Finition de l'interface, états d'erreur, affichage adaptatif, cas limites, test du « pire jour » et reprise des textes.
	13 h–17 h	<code>tsc -noEmit</code> propre des deux côtés, README, <code>docs/api.md</code> , diagramme, seed final.
	17 h–21 h	Installation de zéro sur une machine vierge en suivant votre propre README, puis déroulement complet du scénario de validation.
	21 h–23 h 59	Rédaction de <code>REMISE.md</code> , derniers commits, vérification que le dépôt est public, dépôt sur Teams.

Conseil de survie

L'erreur la plus fréquente consiste à coder soixante heures puis à découvrir, le dernier soir, que l'interface et l'API ne communiquent pas. Branchez un premier appel Axios réel sur un endpoint réel **dès le vendredi après-midi**, même s'il ne renvoie qu'une liste vide. La deuxième erreur la plus fréquente : ne jamais tester l'installation ailleurs que sur la machine où tout a été développé. Le dimanche après-midi, clonez votre propre dépôt dans un dossier neuf et suivez votre README à la lettre, sans rien deviner.

7. Utilisation de l'intelligence artificielle

Autorisé. Poser des questions sur des concepts, des méthodes ou des messages d'erreur à un assistant conversationnel (ChatGPT, Claude, Gemini), consulter la documentation officielle, chercher sur Stack Overflow.

Strictement interdit. Générer le projet ou des fichiers entiers avec un assistant intégré à l'éditeur (GitHub Copilot, Cursor, Windsurf, agents de code), et remettre du code que vous ne pouvez pas expliquer.

Vérification. Il n'y a pas de présentation orale, mais le professeur se réserve le droit de **convoquer un membre individuellement** et de lui demander d'expliquer une portion de code qu'il a committée, ou d'y apporter une petite modification en direct. Un membre incapable d'expliquer son propre code obtient zéro pour la portion concernée. L'historique Git, notamment l'apparition soudaine de volumes importants de code en un seul commit, fait partie des indices examinés.

A. Gabarit du fichier de remise

Copiez ce contenu dans un fichier nommé REMISE.md, remplacez chaque valeur entre chevrons, committez-le à la racine du dépôt et déposez-le sur Teams. Le gabarit est aussi fourni tel quel avec cet énoncé, sous le nom REMISE-template.md.

```
# Remise - Hackathon MindHarbor

**Cours **: Service Web - Groupe 25604 - Session Ete 2026
**Equipe **: <nom de l'equipe>
**Date de remise **: <AAAA-MM-JJ HH:MM>

## 1. Depot GitHub

- **URL (public) **: <https://github.com/xxx/yyy>
- **Commit final a corriger **: <hash complet du commit>
- **Branche **: main
- [ ] Verifie en navigation privée : le depot est bien PUBLIC.

## 2. Membres de l'equipe

| # | Prenom | Nom | Courriel | Compte GitHub |
|---|---|---|---|---|
| 1 | <prenom> | <nom> | <courriel> | <pseudo> |
| 2 | <prenom> | <nom> | <courriel> | <pseudo> |
| 3 | <prenom> | <nom> | <courriel> | <pseudo> |

Capitaine : <prenom nom>

## 3. Comptes de demonstration

| Role | Courriel | Mot de passe | Particularite |
|---|---|---|---|
| Administrateur | <courriel> | <mdp> | - |
| Modérateur | <courriel> | <mdp> | modere le groupe <nom> |
| Utilisateur | <courriel> | <mdp> | 30 jours de journal |
| Utilisateur | <courriel> | <mdp> | profil prive |

## 4. Etat du projet

**Noyau obligatoire **:

| Fonctionnalite | Etat | Remarque |
|---|---|---|
| Journal de bien-etre | complet / partiel / absent | |
| Analyse et tendances | | |
| Ressources et favoris | | |
| Groupes de soutien | | |
| Messagerie et confidentialite | | |
```

```

| Profils et visibilite | | |
| Tableau de bord | | |
| Administration | | |

**Extensions realisees :** <liste, ou "aucune">

**Non termine / limitations connues :** <soyez honnetes et precis>

## 5. Notre part de creativite

<Trois a dix lignes : ce que vous avez ajoute ou soigne de votre propre
initiative, et pourquoi ce choix sert les personnes qui utiliseront
MindHarbor.>

## 6. Verifications avant depot

- [ ] npx tsc --noEmit passe sans erreur dans server/ et dans client/
- [ ] Le projet s'installe et démarre en suivant le README, sur une machine vierge
- [ ] La base Neon est peuplée et accessible
- [ ] Aucun fichier .env n'est commite ; les .env.example sont presents
- [ ] Le scenario de validation de l'enonce a ete deroule en entier

```

B. Exemple de fichier de remise rempli

Voici le même gabarit, rempli par une équipe fictive. Il vous donne le niveau de détail attendu, en particulier aux sections 4 et 5.

```

# Remise - Hackathon MindHarbor

**Cours :** Service Web - Groupe 25604 - Session Ete 2026
**Equipe :** Les Ancres
**Date de remise :** 2026-08-16 21:40

## 1. Depot GitHub

- **URL (public) :** https://github.com/les-ancres/mindharbor
- **Commit final a corriger :** 7f3c1a9e42b6d05c8a1f9e3b7c2d4a6f8e0b1c3d
- **Branche :** main
- [x] Verifie en navigation privee : le depot est bien PUBLIC.

## 2. Membres de l'equipe

| # | Prenom | Nom | Courriel | Compte GitHub |
|---|---|---|---|---|
| 1 | Amina | Belhadj | amina.belhadj@etu.cmaisonneuve.qc.ca | aminabel |
| 2 | Gabriel | Tremblay | gabriel.tremblay@etu.cmaisonneuve.qc.ca | gtremblay-dev |
| 3 | Wei | Chen | wei.chen@etu.cmaisonneuve.qc.ca | weichen42 |
| 4 | Sofia | Rossi | sofia.rossi@etu.cmaisonneuve.qc.ca | sofiarossi |

Capitaine : Amina Belhadj

## 3. Comptes de demonstration

| Role | Courriel | Mot de passe | Particularite |
|---|---|---|---|
| Administrateur | admin@mindharbor.ca | Admin2026! | file de signalements pre-remplie |
| Modérateur | claire@example.com | Modérateur1! | modere "Apprivoiser l'anxiete sociale" |
| Utilisateur | lea@example.com | Utilisateur1! | 30 jours de journal, tendances visibles |
| Utilisateur | tom@example.com | Utilisateur1! | profil prive, messagerie desactivee |

## 4. Etat du projet

**Noyau obligatoire :**

| Fonctionnalite | Etat | Remarque |
|---|---|---|
| Journal de bien-etre | complet | contrainte @@unique([userId, date]) en base |
| Analyse et tendances | complet | 7/30/90 jours + correlation activite-anxiete |
| Ressources et favoris | complet | suggestion contextuelle si anxiete >= 4 |
| Groupes de soutien | complet | groupes prives avec demandes d'adhesion |

```

```
| Messagerie et confidentialite | complet | 3 niveaux de contact verifiés serveur |
| Profils et visibilité | complet | 3 niveaux de visibilité appliqués en select |
| Tableau de bord | complet | resume 7 jours + suggestion du jour |
| Administration | partiel | signalements et stats OK ; la création de
ressources officielles se fait via Prisma Studio, pas encore via l'interface |
```

****Extensions réalisées :** badges de régularité (7/30/100 jours), **export** PDF du journal avec les graphiques.

****Non terminée / limitations connues :**

- Création de ressources par l'admin : API prête (POST /ressources), interface non branchée, faute de temps.
- La pagination du fil de discussion est en "charger plus", pas en pages numérotées.
- Le blocage d'un utilisateur empêche l'envoi mais n'efface pas l'historique de conversation ; c'est un choix assumé, documenté dans le README.

5. Notre part de créativité

Nous avons ajouté un "mode doux" : quand les trois dernières entrées indiquent une humeur inférieure ou égale à 2, l'application réduit d'elle-même le nombre d'éléments affichés sur le tableau de bord, masque les badges et les statistiques de régularité, et remonte les ressources d'apaisement en premier. L'idée vient d'un constat simple : un tableau de bord chargé de chiffres est décourageant les jours difficiles, et ce sont justement ces jours-là qu'il faut demander le moins d'effort à la personne.

Nous avons aussi réécrit tous les états vides à la main. Après une semaine sans entrée, l'application n'affiche pas "0 entrée" mais "Content de vous revoir. On reprend aujourd'hui, si vous voulez." Et le bouton d'aide d'urgence est présent dans l'en-tête de chaque page, pas seulement dans le pied de page.

6. Vérifications avant dépôt

- [x] `npx tsc --noEmit` passe sans erreur dans `server/` et dans `client/`
- [x] Le projet s'installe et démarre en suivant le README, sur une machine vierge
- [x] La base Neon est peuplée et accessible
- [x] Aucun fichier `.env` n'est commité ; les `.env.example` sont présents
- [x] Le scénario de validation de l'énoncé a été déroulé en entier

C. Démarrage rapide

```
# --- Backend (TypeScript) ---
mkdir server && cd server
npm init -y
npm pkg set type=module
npm i express cors helmet morgan dotenv bcryptjs jsonwebtoken zod @prisma/client
npm i -D typescript tsx prisma @types/node @types/express @types/cors
npm i -D @types/morgan @types/bcryptjs @types/jsonwebtoken
npx tsc --init
npx prisma init --datasource-provider postgresql
# coller la chaîne Neon dans .env, puis :
npx prisma migrate dev --name init
npx prisma generate

# --- Frontend (TypeScript) ---
cd ..
npm create vite@latest client -- --template react-ts
cd client && npm i axios react-router-dom recharts
echo "VITE_API_URL=http://localhost:3000/api/v1" > .env
```

Scripts attendus dans `server/package.json` :

```
{
  "scripts": {
    "dev": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js",
    "typecheck": "tsc --noEmit",
  }
}
```

```

    "seed": "tsx prisma/seed.ts",
    "migrate": "prisma migrate dev",
    "studio": "prisma studio"
  },
  "prisma": { "seed": "tsx prisma/seed.ts" }
}

```

D. Squelettes de référence

Listing 7 – server/src/app.ts

```

import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import authRoutes from './routes/auth.routes.js';
import journalRoutes from './routes/journal.routes.js';
import { errorHandler } from './middlewares/error.js';

const app = express();

app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json());
app.use(morgan('dev'));

app.get('/api/v1/health', (_req, res) => res.json({ status: 'ok' }));
app.use('/api/v1/auth', authRoutes);
app.use('/api/v1/journal', journalRoutes);
// ... autres routeurs

app.use((_req, res) => {
  res.status(404).json({ error: { code: 'NOT_FOUND', message: 'Route introuvable.' } });
});
app.use(errorHandler); // toujours en dernier

export default app;

```

Listing 8 – server/src/middlewares/error.ts

```

import type { ErrorRequestHandler } from 'express';

export type FieldError = { field: string; message: string };

export class AppError extends Error {
  constructor(
    public status: number,
    public code: string,
    message: string,
    public details?: FieldError[],
  ) {
    super(message);
  }
}

export const errorHandler: ErrorRequestHandler = (err, _req, res, _next) => {
  const isApp = err instanceof AppError;
  const status = isApp ? err.status : 500;
  if (status === 500) console.error(err);
  res.status(status).json({
    error: {
      code: isApp ? err.code : 'INTERNAL_ERROR',
      message: status === 500 ? 'Une erreur interne est survenue.' : err.message,
      ...(isApp && err.details ? { details: err.details } : {}),
    },
  });
};

```

Listing 9 – server/src/middlewares/auth.ts

```
import type { RequestHandler } from 'express';
import type { Role } from '@prisma/client';
import jwt from 'jsonwebtoken';
import { AppError } from '../error.js';

type AccessPayload = { id: string; role: Role };

export const requireAuth: RequestHandler = (req, _res, next) => {
  const header = req.headers.authorization ?? '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(new AppError(401, 'UNAUTHENTICATED', 'Token manquant.'));
  try {
    req.user = jwt.verify(token, process.env.JWT_ACCESS_SECRET!) as AccessPayload;
    next();
  } catch {
    next(new AppError(401, 'INVALID_TOKEN', 'Token invalide ou expire.'));
  }
};

export const requireRole =
  (...roles: Role[]): RequestHandler =>
  (req, _res, next) =>
    req.user && roles.includes(req.user.role)
      ? next()
      : next(new AppError(403, 'FORBIDDEN', 'Acces refuse.'));
```

Listing 10 – server/src/middlewares/validate.ts – validation zod reutilisable

```
import type { RequestHandler } from 'express';
import type { ZodSchema } from 'zod';
import { AppError } from '../error.js';

export const validateBody =
  (schema: ZodSchema): RequestHandler =>
  (req, _res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      const details = result.error.issues.map((i) => ({
        field: i.path.join('.'),
        message: i.message,
      }));
      return next(new AppError(422, 'VALIDATION_ERROR', 'Donnees invalides.', details));
    }
    req.body = result.data; // req.body est désormais valide et nettoye
    next();
  };
};
```

Listing 11 – server/src/utils/paginate.ts – pagination generique

```
import type { Request } from 'express';

export type Meta = { page: number; limit: number; total: number; totalPages: number };
export type Paginated<T> = { data: T[]; meta: Meta };

export function parsePagination(query: Request['query'], maxLimit = 100) {
  const page = Math.max(1, Number.parseInt(String(query.page ?? '1'), 10) || 1);
  const limit = Math.min(
    maxLimit,
    Math.max(1, Number.parseInt(String(query.limit ?? '20'), 10) || 20),
  );
  return { page, limit, skip: (page - 1) * limit, take: limit };
}

export function buildMeta(page: number, limit: number, total: number): Meta {
  return { page, limit, total, totalPages: Math.ceil(total / limit) };
}
```

E. Pièges classiques

- `new PrismaClient()` instancié dans plusieurs fichiers : le pool de connexions Neon sature et les erreurs deviennent aléatoires.
- CORS oublié : toutes les requêtes Axios échouent, sans message clair.
- `select` Prisma absent sur `User` : le hachage du mot de passe part dans les réponses de l'API.
- `any` semé partout pour faire taire le compilateur : vous perdez l'intérêt de TypeScript et les points qui vont avec.
- Oubli des paquets `@types/...` : le projet ne compile pas et la moitié de la nuit y passe. Installez-les en même temps que la librairie.
- Requête N+1 : boucler sur les entrées de journal pour aller chercher les activités une par une, au lieu d'un `include`.
- Jeton stocké mais jamais rafraîchi : l'utilisateur est déconnecté au bout de quinze minutes.
- Calculs de tendances effectués dans React : contournables par le client, non testables, et lents dès que l'historique grandit.
- Base Neon en veille ou supprimée après la remise : le correcteur ne voit rien fonctionner.
- Seed non réexécutable : le relancer fait exploser les contraintes d'unicité.
- Dates stockées en chaînes de caractères : tri et filtres incohérents. Utilisez `DateTime` et normalisez la date du journal à minuit.
- Dépôt laissé en privé : votre travail est invisible, et la pénalité est lourde.

F. Liste de vérification avant la remise

TypeScript

- ☐ `npm tsc --noEmit` passe sans erreur dans `server/` et dans `client/`.
- ☐ `"strict": true` dans les deux `tsconfig.json`, aucun `@ts-nocheck`.
- ☐ Aucun fichier `.js` ou `.jsx` écrit à la main dans `src/`.

Serveur

- ☐ `git clone, npm install, .env, migrate deploy, seed, npm run dev` fonctionne sur une machine vierge.
- ☐ `GET /api/v1/health` répond.
- ☐ Aucune réponse de l'API ne contient de hachage de mot de passe.
- ☐ Les routes protégées renvoient 401 sans jeton et 403 avec un rôle insuffisant.
- ☐ L'entrée de journal d'un autre utilisateur est inaccessible, y compris pour un administrateur.
- ☐ Une seconde entrée de journal pour la même date est refusée par la base.
- ☐ Toutes les listes paginent et respectent le format `{ data, meta }`.
- ☐ Toutes les erreurs respectent le format `{ error: { code, message } }`.
- ☐ Le dossier `prisma/migrations/` est présent dans le dépôt.

Client

- ☐ Aucune URL d'API en dur : tout passe par `VITE_API_URL`.
- ☐ Aucun appel `fetch()` vers votre API.
- ☐ Le rafraîchissement automatique du jeton fonctionne : testez en réduisant la durée de vie à 30 secondes.
- ☐ Rafraîchir la page sur une route protégée ne déconnecte pas l'utilisateur.

- ☐ Chaque page possède un état de chargement, un état d'erreur et un état vide rédigé avec soin.
- ☐ Les ressources d'aide d'urgence sont accessibles depuis toutes les pages.
- ☐ Interface testée à 375 pixels de large.
- ☐ Le test du « pire jour » a été fait en équipe, et les textes ont été repris.

Dépôt et remise

- ☐ Le dépôt GitHub est **public** (vérifié en navigation privée).
- ☐ `.env` absent du dépôt, `.env.example` présent et complet.
- ☐ `node_modules/` absent du dépôt.
- ☐ README complet, avec les comptes de démonstration.
- ☐ Scénario de validation déroulé en entier, base Neon peuplée et accessible.
- ☐ `REMISE.md` rempli, commité, et **déposé sur Teams** avant dimanche 16 août, 23 h 59.

Bonne chance à toutes les équipes.

Faites du bon code, et faites-le pour quelqu'un.